

PA1, segmentacao de instancias com as arquiteturas da aula

Documento de apoio da apresentacao. Cada numero aqui aponta pro arquivo em `results/` de onde ele saiu, e todo arquivo em `results/` sai de um comando que esta no README.

O que o PA pede e o que a gente entendeu dele

A aula de segmentacao semantica gastou o slide 4 fazendo a distincao entre rotulo class-aware e rotulo instance-aware, e depois passou oitenta slides tratando so do primeiro caso. O PA e a outra metade: fazer as mesmas arquiteturas (encoder-decoder, FCN, SegNet, U-Net, ResUNet, DeepLab, PSPNet) produzirem rotulo instance-aware sem detector com proposta de regioao.

O ponto que demorou pra gente entender e que nao se trata de trocar de arquitetura. A mesma U-Net resolve os dois casos. O que muda sao tres coisas acopladas, e as tres sao decisao de projeto nossa:

1. o que a rede preve, ou seja, a representacao de saida
2. qual perda otimiza aquilo
3. como a previsao vira objeto, ou seja, o pos-processamento

A tese que a gente defende e que a decisao 1 (o que a rede preve) e a que manda, e que a 2 (a perda) e ajuste fino em cima. A evidencia principal e o experimento de oraculo mais adiante, que da pra rodar antes de treinar qualquer rede.

Uma ressalva que a gente so descobriu medindo, e que vale dizer logo: no dataset sintetico o gargalo e 100% a representacao, mas no DSB2018 real a rede tambem limita. O modelo da Parte 1 tira 0.454 de mAP contra um teto de 0.766 do proprio decodificador dele. Entao no real as duas coisas estao apertando ao mesmo tempo, e a gente escreveu a analise nesse sentido em vez de forcar a tese mais simples.

Dataset e split

Opcao A, DSB2018 / BBBC038v1. Sao 670 imagens de treino, cada nucleo num PNG separado, sem sobreposicao. Escolhemos ela porque baixa direto do Broad sem conta, nao tem COCO pra parsear e, principalmente, porque nucleo encostado e a regra e nao a excecao, que e exatamente o problema que o PA quer atacado.

O enunciado exige split estratificado por modalidade. O DSB2018 nao traz essa coluna, entao a gente construiu um proxy: descreve cada imagem com oito estatisticas de cor (media e desvio do cinza, saturacao media, mediana do cinza, fracao de pixel muito escuro, fracao de pixel muito claro, e as diferencas R-G e B-G) e roda um k-means com 4 clusters. Depois o split e feito dentro de cada cluster. Reprodz com `uv run python scripts/split_report.py`, e o resultado esta em `results/split_report.json`.

Os clusters saem interpretaveis, o que e o que da confianca de que o proxy funciona:

cluster	n	cinza medio	saturacao	frac escuro	frac claro	nucleos/img	diam mediano	leitura
0	449	0.041	0.000	0.992	0.000	24.1	22.8 px	fluorescencia, fundo preto
1	70	0.647	0.350	0.000	0.005	36.8	17.4 px	histologia corada, roxo/rosa
2	54	0.771	0.110	0.001	0.720	63.5	17.9 px	brightfield, fundo claro
3	97	0.103	0.000	0.906	0.006	81.4	18.3 px	fluorescencia densa

Duas observacoes que valem na apresentacao. Primeira, o k-means separou modalidade e tambem regime de densidade: os clusters 0 e 3 sao os dois fluorescencia em escala de cinza, mas um tem 24 nucleos por imagem e o outro tem 81. Isso importa porque a Parte 1 pede exatamente a relacao entre desempenho e densidade, e sem estratificar o cluster 3 poderia cair quase todo de um lado do split. Segunda, o cluster 0 sozinho e 67% do dataset, entao um split aleatorio provavelmente ficaria razoavel por sorte, mas os clusters 1 e 2 tem 70 e 54 imagens e sao os que corriam risco de verdade.

O split resultante e 469 treino, 100 validacao, 101 teste. A maior diferenca de proporcao de cluster entre treino e teste ficou em **0.006**, ou seja, a estratificacao funcionou.

cluster	treino	val	teste	% treino	% teste
0	314	67	68	0.670	0.673
1	49	10	11	0.104	0.109
2	38	8	8	0.081	0.079
3	68	15	14	0.145	0.139

A metrica de instancia, e por que a gente teve que definir ela

O slide 6 define AP como precisao media sobre as classes. Isso e a metrica semantica e nao serve aqui: ela nao tem nocao de objeto individual. O PA manda generalizar pro nivel de instancia e proibe usar AP de biblioteca, entao o matching e escrito por nos em <src/pa1/metrics/instance.py>.

A definicao que a gente adotou, que e a convencao do proprio Data Science Bowl 2018: para cada limiar de IoU t em $\{0.50, 0.55, \dots, 0.95\}$, conta cada instancia prevista com no maximo uma instancia verdadeira. Par com IoU maior ou igual a t e TP, previsao sem par e FP, verdade sem par e FN, e a precisao naquele limiar e

$$P(t) = TP / (TP + FP + FN)$$

O AP da imagem e a media de $P(t)$ sobre os dez limiares, e o mAP do conjunto e a media dos AP das imagens. Vale registrar que esse denominador nao e o de precisao no sentido usual (que seria $TP / (TP + FP)$): ele

penaliza FN junto, o que na pratica torna a metrica bem mais dura e e o motivo de os numeros absolutos parecerem baixos.

A regra de matching e greedy por IoU decrescente, que e o item 4 da Parte 1. Ordena todos os pares candidatos com IoU maior ou igual ao limiar por IoU decrescente e vai casando, pulando quem ja foi usado. A alternativa esta implementada tambem (`--matching hungarian`, que resolve a atribuicao global com `linear_sum_assignment` e so depois corta os pares abaixo do limiar) e a apresentacao mostra as duas lado a lado. Guloso e mais duro em casos ambiguos porque uma escolha localmente boa pode roubar o par de outra instancia, mas em quase toda imagem real os dois coincidem, porque um nucleo bem previsto tem IoU alto com um unico nucleo verdadeiro e proximo de zero com todos os outros.

A implementacao da matriz de IoU e vetorizada. Em vez de dois lacos sobre instancias, a gente codifica cada par de labels em `pred * (n_gt + 1) + gt` nos pixels onde os dois tem rotulo, conta com `bincount` e reformata pra matriz. Os testes em `tests/test_instance_metrics.py` conferem a conta na mao: match perfeito da AP 1, um FN da 0.5, um FP espurio da 2/3, um caso construido de IoU 8/16 bate, e dois objetos fundidos num blob so dao menos de 0.5.

Parte 0, o teste unitario sintetico

O gerador esta em `src/pa1/data/synthetic.py`. Imagens 128x128, de 5 a 20 elipses, raio de 5 a 18 px, angulo e razao de eixos aleatorios, intensidade por instancia, nivel de fundo, borramento e ruido variaveis. Nao guarda nada em disco: cada indice deriva uma seed, entao o dataset e deterministico e os splits ficam disjuntos so mudando a seed base.

O detalhe que importa e que **as elipses tem que encostar**. A primeira versao sorteava posicao uniforme e quase nenhuma encostava, o que tornava o teste inutil (componentes conexos resolveria tudo e a Parte 1 nao teria fracasso nenhum pra mostrar). A versao final ancora 70% das elipses novas ao lado de uma ja colocada, com a distancia entre centros em torno da soma dos raios medios.

Dois bugs nossos apareceram nesse caminho e valem ser contados. O primeiro: elipse desenhada por cima enterra a anterior, e a imagem podia acabar com menos de 5 instancias, fora da faixa que o PA pede. Arrumamos contando quantas instancias sobrevivem a cada passo em vez de quantas foram desenhadas. O segundo so apareceu quando fomos medir o teto de oraculo, e esta descrito na secao seguinte.

O experimento de oraculo, que e o argumento central do trabalho

Antes de treinar qualquer rede, da pra responder a pergunta "o problema esta na rede ou na representacao?" alimentando o pos-processamento com a saida perfeita. Se a rede acertasse tudo, quanto cada decodificacao entregaria?

Isso e `scripts/oracle_ceiling.py`, que nao treina nada e nao carrega checkpoint. Rodamos nos dois datasets, no split de teste inteiro. Saida em `results/oracle_ceiling_synthetic_test.json` e `results/oracle_ceiling_dsb2018_test.json`.

decodificacao	entrada perfeita	sintetico (128 img)	DSB2018 (101 img)
limiar + componentes conexos (Parte 1)	mascara semantica	0.115	0.766

decodificacao	entrada perfeita	sintetico (128 img)	DSB2018 (101 img)
watershed com marcadores (Parte 2)	fronteira e distancia	0.791	0.960
watershed so do mapa de distancia	so a distancia	0.790	0.970

E so no terco mais denso de cada split, que e onde o problema aparece:

decodificacao	sintetico (>= 16 obj)	DSB2018 (>= 45 obj)
componentes conexos	0.077	0.660
watershed	0.739	0.954

Tres leituras, e a terceira e a que mais vale falar na apresentacao.

Primeira, **o teto do componentes conexos e um teto de verdade**. Com a mascara semantica perfeita, ou seja, com uma rede que nao erra um pixel sequer, o metodo da Parte 1 nao passa de 0.115 no sintetico e 0.766 no DSB2018. Nenhum treino, nenhum encoder, nenhuma perda melhora isso, porque a informacao que separa dois nucleos encostados nao existe numa mascara binaria: eles formam um blob unico e conexo.

Segunda, **trocar o que a rede preve muda o teto, com a mesma arquitetura**. De 0.115 para 0.791 no sintetico e de 0.766 para 0.960 no DSB2018. E no terco mais denso, que e o caso que interessa, de 0.077 para 0.739 e de 0.660 para 0.954.

Terceira, e essa a gente so percebeu depois de rodar: **o dataset sintetico e muito mais duro que o real**, de proposito. No sintetico quase toda elipse encosta em outra, entao componentes conexos e catastrofico (0.115). No DSB2018 boa parte dos nucleos esta isolada, e componentes conexos ja entrega 0.766. Ou seja, o sintetico nao e uma versao facil do problema real, e uma versao concentrada exatamente no modo de falha que a gente quer atacar. Isso vai importar de novo na Parte 2, quando o ganho no real for menor que no sintetico: nao e o metodo funcionando pior, e o problema aparecendo em menor proporcao.

O mesmo experimento em 24 imagens sinteticas esta como teste em

`tests/test_targets_and_postprocess.py`

(`test_watershed_beats_connected_components_with_perfect_input`), entao ele roda no `pytest` e trava se alguem quebrar a geracao de alvo. Nessas 24, em 24 de 24 o componentes conexos funde pelo menos duas elipses.

O bug numero dois apareceu aqui. Na primeira versao do gerador o teto do watershed dava 0.74 em vez de 0.83 nas 24 imagens de teste, e a gente foi investigar imagem por imagem esperando achar um erro no watershed. O culpado eram instancias de 6 a 8 pixels, restos de elipses quase totalmente enterradas por outra desenhada por cima. Lasca de 6 pixels nao e objeto, ninguem casa com ela, e ela puxava a metrica inteira pra baixo. Colocamos um filtro de area minima no gerador e o teto subiu. Vale contar porque e um caso de a metrica estar certa e o **dataset** estar errado, e a gente quase foi mexer no lugar errado.

O teste unitario propriamente dito

O PA pede que treine em menos de 5 minutos e que a metrica seja reportada. Medimos nos dois lugares onde o codigo rodou.

Na GPU (Tesla P100 do Kaggle), com o config do repo (512 imagens de treino, 12 epocas), as duas versoes:

config	tempo de treino	Dice (teste)	mAP (teste)	erro de contagem
<code>synthetic_baseline</code> (binario, Parte 1)	1.2 min	0.9920	0.1032	9.70
<code>synthetic_boundary</code> (trilha A, Parte 2)	1.3 min	0.9857	0.5122	2.22

Na CPU (i7 de 18 nucleos, sem GPU), com 320 imagens e 8 epocas, a versao binaria leva 7.4 minutos no total, mas a metrica satura na quinta epoca, aos **4.8 minutos**:

epoca	minutos	loss	IoU	Dice	mAP
1	1.0	0.847	0.722	0.798	0.046
2	1.9	0.656	0.951	0.974	0.082
3	2.9	0.576	0.886	0.924	0.089
4	3.8	0.513	0.946	0.971	0.093
5	4.8	0.465	0.976	0.988	0.102
6	5.7	0.435	0.977	0.989	0.110
8	7.4	0.409	0.976	0.988	0.100

Dos dois lados o numero pra levar e o mesmo: **Dice em torno de 0.99 e mAP de instancia em torno de 0.10**. A rede resolveu a segmentacao semantica de forma essencialmente perfeita, e o mAP de instancia ficou igual ao teto de oraculo do componentes conexos (0.102), medido antes de treinar. Ou seja, o erro que sobra nao e da rede, e inteiramente do decodificador. Isso justifica a decisao de selecionar checkpoint por mAP de validacao e nao por loss: aqui as duas coisas estao completamente descorreladas.

E a linha de baixo da primeira tabela ja antecipa a Parte 2 no mesmo dataset e com a mesma arquitetura: trocando so o que a rede preve, o mAP vai de **0.103 para 0.512** e o erro de contagem cai de **9.70 para 2.22 nucleos por imagem**. Quase 5 vezes de ganho, com Dice praticamente igual (0.9920 contra 0.9857, ou seja, ligeiramente pior).

Parte 1, o baseline e a quantificacao do fracasso

`configs/dsb2018_baseline.yaml`: U-Net com encoder ResNet34 pre-treinado na ImageNet, decoder nosso com skip connections (slides 25 a 29), saida de 1 canal, perda BCE mais Dice, crop de 256, 40 epocas, AdamW com cosine annealing. Treino levou 5.4 minutos na P100. Instancias saem por limiar 0.5 mais componentes conexos com conectividade 8 e descarte de blobs abaixo de 20 px.

Resultado no split de teste (101 imagens), em `results/parte1/metrics.json`:

metrica	valor
IoU semantico	0.8415

metrica	valor
Dice	0.9120
mAP @[.50:.95]	0.4654
AP @.50	0.6931
erro absoluto de contagem	10.05 nucleos por imagem

(esses numeros sao com a decodificacao padrao, limiar 0.5. Calibrando o limiar na validacao, como a gente faz na Parte 2 pros dois modelos, o baseline sobe pra mAP 0.5351 e erro de contagem 8.57. A comparacao justa esta na Parte 2.)

Por limiar de IoU, que e o que mostra onde a coisa desmonta:

limiar	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95
precisao	0.693	0.663	0.633	0.601	0.572	0.509	0.423	0.314	0.188	0.058

Quanto desse erro e da rede e quanto e do decodificador

Comparando com o teto de oraculo medido antes: o decodificador ingenuo, alimentado com a mascara perfeita, daria 0.766 nesse mesmo split. Esse mesmo modelo, com o limiar calibrado na validacao (ver a Parte 2), entrega 0.535. Entao o erro se reparte quase ao meio:

de onde vem	quanto	por que
culpa da rede	$0.766 - 0.535 = \mathbf{0.231}$	a mascara semantica nao e perfeita
culpa do decodificador	$1.000 - 0.766 = \mathbf{0.234}$	nucleo encostado que nem mascara perfeita separa

Isso e diferente do que acontece no sintetico, onde a rede praticamente satura o teto (0.103 medido contra 0.115 de teto do componentes conexos) e quase todo o erro e do decodificador. E uma nuance que a gente so viu depois de medir os dois, e ela importa pra Parte 2: no DSB2018 a mudanca de representacao ataca metade do problema, nao o problema todo, e por isso o ganho aqui e menor que no sintetico. Nao e o metodo funcionando pior, e ele atacando uma fracao menor do erro.

O numero que mais importa aqui e o erro de contagem: **10 nucleos de erro por imagem**, num dataset onde a mediana e algo em torno de 25 nucleos por imagem. Com Dice de 0.91, ou seja, com a mascara semantica essencialmente certa, o metodo erra a contagem em cerca de 40%. E o mesmo padrao do sintetico, so que menos extremo porque nem todo nucleo do DSB2018 encosta em outro.

A regra de matching, na pratica

Rodamos a avaliacao com as duas regras no mesmo checkpoint e no mesmo split:

regra	mAP	AP50	erro de contagem
guloso por IoU decrescente	0.4654	0.6931	10.05
Hungarian	0.4654	0.6931	10.05

Deram **exatamente o mesmo numero**, ate a quarta casa. Isso confirma o argumento que a gente ia fazer no papel: as duas regras so divergem quando existe ambiguidade real, e nucleo previsto razoavelmente bem tem IoU alto com um unico nucleo verdadeiro e perto de zero com todos os outros, entao a atribuicao otima e a gulosa coincidem. Reportamos as duas justamente porque o enunciado pede a regra explicita, e achamos honesto mostrar que nesse dataset a escolha nao muda nada. Em um dataset com objetos muito sobrepostos a conclusao poderia ser outra.

Parte 2, trilha A: fronteira e watershed

Escolhemos a trilha A. As tres razoes, em ordem de peso:

O pos-processamento e deterministico. A trilha B (embeddings discriminativos) precisa de clustering na inferencia, e mean-shift e DBSCAN trazem hiperparametro (largura de banda, eps, min_samples) que muda o numero de objetos encontrados. Isso vira uma segunda fonte de erro que a gente teria que separar da qualidade da rede, e com dois alunos e prazo de duas semanas era risco demais.

A trilha C (centro mais offsets) supoe implicitamente que o objeto e mais ou menos convexo e que apontar pro centro identifica o objeto. Nucleo em divisao no DSB2018 e frequentemente bilobado, e nesses o centro geometrico as vezes cai fora da mascara.

E a trilha A e a que mais se encaixa nas pecas que a aula deu: a perda de classe e CE balanceada ou focal (slides 73 a 79) e a de distancia e L1 ou L2 (slide 80), tudo material de aula.

O que a rede preve

O encoder-decoder e literalmente o mesmo da Parte 1, so a head de 1x1 muda de 1 canal pra 4:

- canais 0, 1, 2: logits de tres classes, fundo / interior / fronteira entre instancias, passados por softmax
- canal 3: mapa continuo de distancia ao fundo, passado por sigmoid porque o alvo e normalizado em [0,1] por instancia

A normalizacao por instancia do mapa de distancia foi escolha nossa e vale defender: sem ela, o nucleo grande domina a regressao e o pequeno vira ruído numerico. O que a decodificacao precisa e a forma do morro, onde fica o pico, nao a altura absoluta em pixels.

Como gerar o rotulo de fronteira a partir das mascaras individuais

Essa e a primeira pergunta que o enunciado faz na trilha A, e a resposta e o detalhe que faz tudo funcionar. **A erosao e por instancia, nao na mascara semantica.** Para cada nucleo separadamente, o que sobra da erosao vira interior (classe 1) e a casca que a erosao comeu vira fronteira (classe 2).

Se a gente erodisse o foreground inteiro de uma vez, o contato entre dois nucleos encostados ficaria no meio de uma regio de foreground e nao viraria fronteira nenhuma, entao o watershed nao teria onde cortar e a trilha A degeneraria de volta na Parte 1. Erodindo instancia a instancia, no contato existe uma casca de cada lado, e a classe 2 aparece exatamente onde componentes conexos fundiria os dois. Tem um teste so pra isso, [test_boundary_separates_touching_instances](#), que constroi dois retangulos encostados sem um pixel de fundo entre eles e verifica que o interior sai com dois componentes.

O mapa de distancia tem o mesmo cuidado: a EDT roda dentro do bounding box de cada instancia, e nao globalmente. Uma EDT global nao teria vale nenhum no contato entre dois nucleos encostados, e o vale e

justamente o que o watershed usa pra decidir onde cortar. Tambem tem teste (`test_distance_does_not_leak_between_touching_instances`).

Que espessura

Segunda pergunta do enunciado. Espessura fina demais e a rede nao consegue prever, uma casca de 1 px praticamente some no downsample de stride 32 do encoder. Grossa demais come o interior dos nucleos pequenos e eles deixam de virar marcador no watershed.

Medimos o trade-off em 200 imagens de treino e 9367 nucleos, com `uv run python scripts/class_stats.py --config configs/dsb2018_boundary.yaml` (saida em `results/class_stats.json`):

espessura	fundo	interior	fronteira	alpha (1/sqrt f)	marcadores perdidos	marcadores rachados
1	0.868	0.114	0.018	[0.28, 0.77, 1.95]	0	54
2	0.868	0.097	0.035	[0.33, 1.00, 1.67]	0	82
3	0.868	0.081	0.051	[0.36, 1.17, 1.47]	0	137
4	0.868	0.068	0.064	[0.36, 1.30, 1.34]	0	166

"Perdidos" e quantos nucleos ficam sem interior nenhum depois da erosao, ou seja, nao viram marcador e viram falso negativo garantido. "Rachados" e quantos ficam com o interior partido em dois componentes, ou seja, viram dois objetos e produzem falso positivo garantido.

Ficamos com **espessura 2**. Nenhum nucleo perde o marcador em nenhuma das espessuras testadas (o codigo tem um cuidado especifico pra isso: se a erosao apagaria o nucleo inteiro, ele guarda o pico da distancia como interior). O que decide e a outra coluna: rachados cresce monotonicamente, de 54 pra 166, e espessura 2 e o ponto onde a fronteira ja e grossa o bastante pra rede aprender sem estar rachando interior demais. Espessura 1 tem menos rachados mas deixa a classe 2 em 1.8% dos pixels, e nesse regime a rede simplesmente aprende a nunca prever fronteira.

Como pesar a classe fronteira, que e minoritaria

Terceira pergunta. Com espessura 2 a fronteira e **3.5% dos pixels** e o fundo e 86.8%, uma razao de 25 para 1. E o desbalanceamento severo que o enunciado antecipa, e e exatamente o que o eixo 2 da Parte 3 vai medir.

Temos duas alavancas implementadas. A primeira e o peso por classe alpha da CE balanceada (slides 74 e 75), calculado de `alpha_from_frequencies`, que suporta $1/f$ e $1/\sqrt{f}$ com um teto. Usamos $1/\sqrt{f}$ porque $1/f$ puro coloca a fronteira em quase 30 vezes o peso do fundo e a loss vira praticamente so fronteira. A segunda e um mapa de peso por pixel (`touching_weight_map`), que da peso extra so no contato entre duas instancias diferentes, na linha da ideia do mapa de peso do paper original da U-Net. A distincao importa: a casca externa de um nucleo isolado nao separa nada, so a casca entre dois encostados separa.

Por que isso resolve o problema da Parte 1

Em uma frase, que e como a gente vai falar na apresentacao: componentes conexos so tem acesso a mascara binaria, e dois nucleos encostados formam um blob unico e conexo, entao nao ha informacao ali que permita separa-los. A trilha A faz a rede marcar explicitamente a casca de cada nucleo, e o interior que sobra ja vem desconectado entre dois vizinhos. Componentes conexos sobre o interior ja da o numero certo de objetos, e o watershed so precisa crescer cada marcador de volta ate a borda real usando o mapa de distancia como relevo.

Um detalhe de implementacao que custou tempo: a mascara do watershed usa interior mais fronteira, nao so interior. Se usasse so o interior, todos os nucleos sairiam erodidos e o IoU com o ground truth nunca passaria de 0.5, o que zeraria o mAP inteiro mesmo com tudo mais perfeito. Tem teste pra isso tambem ([test_watershed_recovers_the_full_object_not_just_the_interior](#)). O relevo e `-dist` porque o watershed do skimage enche bacias a partir dos minimos, entao o centro do nucleo, onde a distancia e maxima, precisa virar o fundo da bacia.

O resultado, lado a lado com a Parte 1

Mesmo encoder, mesmo decoder, mesmo split, mesmas 40 epocas, mesmo otimizador. Muda so a head de 1x1 (1 canal para 4), a perda e o pos-processamento. Split de teste, 101 imagens, matching guloso, em [results/parte2/metrics.json](#) e [results/parte2/comparacao/](#):

metrica	Parte 1 (limiar + CC)	Parte 2 (fronteira + watershed)	delta
IoU semantico	0.8415	0.8052	-0.0363
Dice	0.9120	0.8881	-0.0239
mAP @[.50:.95]	0.4654	0.4972	+0.0318
AP @.50	0.6931	0.7592	+0.0662
erro de contagem	10.05	4.10	-5.95

Esse e o slide principal da Parte 2, e o que ele mostra e melhor do que so "o mAP subiu". **As duas metricas se movem em direcoes opostas.** O Dice piorou, de 0.9120 para 0.8881, e o IoU semantico tambem. Pela metrica da aula, a Parte 2 e um modelo pior. E ao mesmo tempo o erro de contagem caiu **59%**, de 10.0 nucleos por imagem para 4.1, e o AP no limiar 0.50 subiu quase 7 pontos.

Da pra explicar exatamente por que o Dice piora. A rede da Parte 2 gasta capacidade aprendendo uma casca de 2 px que representa 3.5% dos pixels e que, do ponto de vista semantico, e foreground igual ao interior. Quando ela erra pro lado de marcar fronteira demais, o foreground reconstruido (interior mais fronteira) fica um pouco mais magro que a verdade, e o Dice cai. Do ponto de vista de instancia isso e um preco baixissimo: perder 2 pontos de Dice pra cortar 59% do erro de contagem.

E o motivo de a apresentacao insistir nisso e que o Dice e a metrica que a aula ensinou. Se a gente tivesse selecionado modelo por Dice, teria escolhido o modelo errado. Foi por isso que o loop de treino seleciona checkpoint por mAP de validacao desde o comeco.

Por limiar de IoU, o perfil dos dois:

limiar	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95
Parte 1	0.693	0.663	0.633	0.601	0.572	0.509	0.423	0.314	0.188	0.058
Parte 2	0.759	0.727	0.697	0.659	0.614	0.552	0.461	0.328	0.157	0.018

Esses numeros sao com os limiares de decodificacao no chute, 0.5 pra tudo, que foi como a gente rodou primeiro. A secao seguinte mostra que isso estava deixando bastante desempenho na mesa nos **dois** modelos, e refaz a comparacao direito.

A calibracao da decodificacao, e por que ela quase nos fez errar a analise

Os limiares dos dois pos-processamentos estavam no chute (0.5 pra tudo). Calibramos os dois **no split de validacao** com `scripts/tune_watershed.py`, que roda a rede uma vez e varre os limiares em numpy, e so depois medimos no teste. Os melhores foram limiar 0.8 e min_size 20 pro baseline, e interior 0.5, foreground 0.7 e min_size 20 pra trilha A.

decodificacao	Parte 1	Parte 2
padrao (limiar 0.5)	0.4654	0.4972
calibrada na validacao	0.5351	0.5787

Os dois ganham muito, cerca de 0.07 e 0.08 de mAP, **sem tocar em um peso sequer da rede**. Os dois modelos preferem um limiar de foreground bem mais alto que 0.5, o que quer dizer que eles estao sistematicamente prevendo foreground demais, e cortar mais fundo melhora o IoU de cada instancia.

Aqui a gente quase errou feio. Depois de calibrar so a Parte 2, ela passava a ganhar nos dez limiares de IoU, e a leitura obvia era "o deficit da Parte 2 em IoU alto era artefato de hiperparametro". So que essa comparacao era injusta: um lado calibrado contra o outro no chute. Calibrando os dois, o padrao volta a aparecer, e ele e real. Fica de licao: calibrar so o metodo que a gente esta defendendo e uma forma facil de se enganar.

O resultado final, com os dois lados calibrados

Split de teste, 101 imagens, matching guloso, os dois com a decodificacao escolhida na validacao:

metrica	Parte 1 (limiar + CC)					Parte 2 (fronteira + watershed)					delta
IoU semantico	0.8415					0.8052					-0.0363
Dice	0.9120					0.8881					-0.0239
mAP @[.50:.95]	0.5351					0.5787					+0.0436
AP @.50	0.7312					0.8137					+0.0825
erro de contagem	8.57					4.07					-4.50
limiar	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95	
Parte 1	0.731	0.701	0.677	0.650	0.618	0.579	0.524	0.442	0.314	0.114	
Parte 2	0.814	0.790	0.769	0.737	0.695	0.639	0.546	0.444	0.279	0.075	

O padrao e claro e sobrevive a calibracao: **a Parte 2 ganha de 0.50 a 0.85 e perde em 0.90 e 0.95**. E a explicacao original continua de pe. O watershed decide a divisa entre dois nucleos por um criterio geometrico, a linha entre duas bacias do mapa de distancia, e essa linha raramente coincide pixel a pixel com o tracado do anotador humano. Entao a Parte 2 acerta muito mais objetos, cada um com contorno um pouco pior. Componentes conexos, quando por acaso acerta um nucleo isolado, acerta o contorno inteiro, porque ali o contorno e literalmente o limiar da probabilidade.

Como o mAP media dez limiares e sete deles estao na faixa onde a Parte 2 ganha, o saldo e fortemente positivo. E o erro de contagem, que e o que um biologo realmente usaria, cai pela metade, de 8.57 pra 4.07 nucleos por imagem.

No sintetico, onde praticamente todo objeto encosta em outro, o mesmo efeito aparece muito mais forte: mAP de 0.1032 para 0.5122 e erro de contagem de 9.70 para 2.22.

Parte 3, ablacoes nos eixos 2 e 3

Rodamos os eixos 2 (funcao de perda) e 3 (contexto global). Cada configuracao com 2 seeds, reportando media e desvio, como o enunciado exige.

As ablacoes rodam com 20 epocas em vez das 40 do modelo final, e sao 18 treinos ao todo (6 configuracoes no eixo 2 e 3 no eixo 3, cada uma com 2 seeds), 53 minutos de GPU.

Uma armadilha que a gente caiu e vale contar. A ideia inicial era justificar o corte dizendo "na epoca 20 o modelo final ja esta em 0.4766 de mAP contra 0.4780 na epoca 40, entao 20 epocas bastam". Isso esta errado, e a gente so viu comparando os numeros depois. O scheduler e um CosineAnnealingLR com `T_max=epochs`, entao mudar de 40 pra 20 epocas nao corta o treino ao meio, muda a curva inteira de learning rate: com `T_max=20` a taxa ja chegou perto de zero na epoca 20, enquanto com `T_max=40` ela ainda esta na metade. Rodando a mesma configuracao (focal balanceada gamma=2, seed 0) nos dois regimes:

epoca	4	8	12	16	20
dentro do schedule de 40 epocas	0.141	0.300	0.340	0.430	0.477
com schedule de 20 epocas	0.123	0.251	0.315	0.384	0.391

Ou seja, as ablacoes vivem num regime pior, e **os numeros absolutos delas nao sao comparaveis com os do modelo final**. O que continua valendo, e e o que importa aqui, e a comparacao **dentro** de cada eixo: todas as configuracoes de um eixo compartilham o mesmo orcamento e o mesmo schedule, entao a ordem entre elas e legitima. A gente prefere deixar isso escrito a fingir que nao existe.

Deixamos o eixo 1 (como recuperar resolucao) de fora por uma razao de metodo: pool indices, skip connections e atrous nao sao so tres jeitos de fazer upsample, eles mudam a arquitetura junto. Comparar SegNet contra U-Net contra DeepLab mantendo tudo mais igual daria um experimento em que a variavel isolada nao e realmente uma variavel so, e com o orcamento que a gente tinha preferimos dois eixos limpos a tres sujos. Vale dizer que a analise de campo receptivo da Parte 5 acaba tocando no eixo 1 por outro caminho.

Eixo 2, a funcao de perda

O caminho que o enunciado desenha, CE, CE balanceada, focal, focal balanceada (slides 73 a 79), com gamma em {0, 1, 2, 5}. As seis configuracoes:

chave	perda	gamma	alpha
ce	CE	0	sem
ce_bal	CE balanceada	0	[0.45, 0.80, 1.75]
focal_g1	focal	1	sem
focal_g2	focal	2	sem
focal_g5	focal	5	sem
focal_g2_bal	focal balanceada	2	[0.45, 0.80, 1.75]

gamma = 0 e literalmente CE, o código é o mesmo (**FocalLoss** com gamma 0 reduz a CE), o que deixa o eixo contínuo e não um conjunto de perdas diferentes.

O que a gente espera antes de olhar, pra poder confrontar depois: com fundo em 86.8% dos pixels e fronteira em 3.5%, a CE pura deveria aprender a quase nunca prever fronteira, porque errar fronteira custa pouco na soma. Alpha ataca isso por classe e gamma ataca por dificuldade. A pergunta interessante é se as duas alavancas somam ou se uma anula a outra, que é o que a linha focal balanceada responde.

Eixo 3, contexto global

Os dois mecanismos dos slides 51 a 55, plugados no mesmo ponto da rede (topo do encoder, antes do decoder), variando só o módulo. Estão em [src/pa1/models/context.py](#).

Sem contexto é o braço de controle. ParseNet (image pooling, slides 51 e 52) tira a média global do feature map, projeta com 1x1, faz broadcast e concatena, então cada pixel passa a conhecer a estatística da imagem inteira. PSPNet (pyramid pooling, slides 53 a 55) faz pooling em várias grades (1x1, 2x2, 3x3, 6x6), projeta cada nível e concatena todos, o que dá contexto em várias escalas em vez de só global.

A pergunta específica do enunciado é boa e a gente montou a tabela pra responder ela diretamente:

contexto global ajuda a separar instancias, ou só a classifica-las melhor? Por isso a tabela do eixo 3 reporta Dice (que mede classificar, ou seja, achar o objeto) do lado de mAP e erro de contagem (que medem separar). Se contexto só ajudasse a classificar, o Dice subiria e o mAP ficaria parado.

A intuição previa, que a gente vai confrontar com o número: separar dois núcleos encostados é uma decisão sobre dois ou três pixels de contato, uma decisão local. Média global da imagem inteira não carrega informação sobre onde exatamente cortar. Então a expectativa é que contexto ajude pouco no mAP. Isso conversa com a análise da Parte 5.

Resultado do eixo 2

Split de validação, média e desvio sobre as seeds 0 e 1, em [results/parte3/eixo2_results.json](#):

configuracao	mAP	Dice	erro de contagem
CE (gamma=0)	0.4879 +- 0.023	0.8929 +- 0.007	4.96 +- 0.11
CE balanceada	0.4189 +- 0.004	0.8613 +- 0.002	4.20 +- 0.34
focal gamma=1	0.4907 +- 0.020	0.8867 +- 0.010	5.00 +- 0.13

configuracao	mAP	Dice	erro de contagem
focal gamma=2	0.4927 +- 0.000	0.8870 +- 0.006	4.59 +- 0.33
focal gamma=5	0.3965 +- 0.025	0.8392 +- 0.015	4.98 +- 0.06
focal balanceada gamma=2	0.3913 +- 0.012	0.8441 +- 0.001	4.40 +- 0.07

Tres coisas, e a primeira contraria o que a gente tinha escrito antes de rodar.

Balancear piora, e nao e pouco. CE cai de 0.4879 pra 0.4189 quando entra o alpha, e focal gamma=2 cai de 0.4927 pra 0.3913. Sao 0.07 e 0.10 de mAP, muito acima do desvio entre seeds. A gente tinha previsto o contrario, com o argumento de que a fronteira e 3.5% dos pixels e a CE pura ia ignorar ela.

A explicacao que a gente defende, e que bate com o resto do trabalho: dar peso 1.75 pra fronteira faz a rede marcar fronteira **demais**, nao apenas o suficiente. A casca prevista engorda, o interior mais fronteira reconstruido fica mais magro que o nucleo real, e o IoU de cada instancia casada cai. A coluna do Dice confirma: ela cai junto (0.8929 para 0.8613 na CE, 0.8870 para 0.8441 na focal). Ou seja, o alpha nao esta corrigindo desbalanceamento, esta desbalanceando pro outro lado.

O detalhe que fecha o argumento e o **erro de contagem indo na direcao oposta**: com alpha ele melhora (4.96 para 4.20 na CE, 4.59 para 4.40 na focal). Faz sentido, porque fronteira mais grossa separa melhor. Entao o alpha faz exatamente o que se esperava dele, separar melhor, e o preco em delineamento e maior que o ganho. E o mesmo trade-off entre separar e delinear que apareceu na Parte 2 e que vai reaparecer na correcao da Parte 5. Se a metrica fosse so contagem de celulas, que e o que um biologo normalmente quer, a escolha seria a oposta.

gamma quase nao importa entre 0 e 2. 0.4879, 0.4907 e 0.4927 para gamma 0, 1 e 2, com desvio de ate 0.023. As tres empatam. So gamma=5 quebra (0.3965), e ai a explicacao e a usual: com gamma tao alto quase todo pixel vira "facil" e o gradiente some.

O desvio entre seeds e pequeno, no maximo 0.025, o que da confianca de que as diferencas de 0.07 e 0.10 acima sao reais e nao ruido.

Resultado do eixo 3

Mesmo protocolo, variando so o modulo de contexto no topo do encoder. Em

[results/parte3/eixo3_results.json](#):

configuracao	mAP	Dice	erro de contagem
sem contexto	0.3895 +- 0.012	0.8444 +- 0.007	4.45 +- 0.27
image pooling (ParseNet)	0.4253 +- 0.019	0.8584 +- 0.012	4.53 +- 0.20
pyramid pooling (PSPNet)	0.4097 +- 0.024	0.8532 +- 0.015	3.96 +- 0.19

Antes de interpretar, uma ressalva de metodo: o eixo 3 roda em cima da perda do config original, que e a focal balanceada, e o eixo 2 mostrou depois que ela e uma das piores. Por isso o braco "sem contexto" aqui esta em 0.3895 e nao perto de 0.49. As tres barras compartilham essa perda, entao a comparacao entre elas continua valendo, mas o patamar todo esta rebaixado.

A resposta pra pergunta do enunciado, que era se contexto global ajuda a separar instancias ou so a classifica-las melhor: **majoritariamente a classificar**.

Os dois modulos sobem o Dice de forma consistente, +0.014 no ParseNet e +0.009 no PSPNet. Ou seja, os dois ajudam a decidir se um pixel e nucleo ou fundo, que e classificacao. Mas no erro de contagem, que e a medida direta de separacao, o ParseNet **piora** (4.45 para 4.53) e so o PSPNet melhora (4.45 para 3.96, 11%).

Isso conversa exatamente com a analise de campo receptivo da Parte 5. Media global da imagem inteira, que e o que o ParseNet faz, e um unico vetor por imagem: ele diz "isso aqui e uma lamina de fluorescencia escura" e ajuda a calibrar o limiar de foreground, mas nao carrega nenhuma informacao sobre **onde** cortar entre dois nucleos vizinhos, porque essa e uma decisao sobre dois ou tres pixels de contato. Ja o PSPNet faz pooling em grades 2x2, 3x3 e 6x6 alem da global, e essas grades ainda tem alguma localizacao, o que explica ele ser o unico que mexe no erro de contagem.

Resumindo pra apresentacao: contexto global melhora o mAP, mas por classificar melhor, nao por separar melhor. Quem separa melhor e o contexto **multi-escala**, e mesmo assim modestamente.

O que a Parte 3 mudou no modelo final, e por que quase nada mudou

As ablacoes nao ficaram como apendice: a gente levou as conclusoes delas de volta pro modelo final e mediu. Duas mudancas, testadas em `configs/dsb2018_final.yaml` (sem alpha) e `configs/dsb2018_final_ctx.yaml` (sem alpha mais image pooling), as duas treinadas com as 40 epocas do modelo final e as duas com a decodificacao calibrada na propria validacao.

O criterio de escolha e o mAP de validacao, cada modelo no seu proprio otimo de decodificacao:

modelo	perda	contexto	mAP de validacao
<code>dsb2018_boundary</code> (o original)	focal gamma=2 com alpha	nenhum	0.5612
<code>dsb2018_final</code>	focal gamma=2 sem alpha	nenhum	0.5344
<code>dsb2018_final_ctx</code>	focal gamma=2 sem alpha	image pooling	0.5539

Tirar o alpha nao ajudou no orcamento cheio, apesar de ter ajudado no orcamento das ablacoes. No regime de 20 epocas a diferenca entre com e sem alpha era de 0.10 de mAP a favor de tirar. Com 40 epocas e o schedule completo, a diferenca inverte e fica em 0.027 a favor de manter.

Esse e um resultado negativo e a gente vai apresentar ele como tal, porque ele e mais informativo que o positivo teria sido. A leitura: **a conclusao da ablacao nao transferiu para o regime do modelo final**. E consistente com a armadilha do scheduler que a gente ja tinha detectado. O alpha empurra a rede a marcar fronteira demais, o que atrapalha cedo no treino, mas com learning rate alto por mais tempo a rede tem folga pra desfazer esse vies e acaba aproveitando o sinal extra na classe minoritaria. Ou seja, o alpha nao e ruim, ele e **lento**, e o orcamento de 20 epocas nao dava tempo de ele pagar.

O que isso custa: nao da pra usar uma ablacao barata como substituta de um experimento no regime real, mesmo que ela seja limpa, com 2 seeds e desvio pequeno. O ranking que ela produz vale dentro do orcamento dela, e so.

O image pooling ficou no meio (0.5539) e tambem nao superou o original, mas ele reproduz de novo o achado do eixo 3 de forma bem nitida: e o modelo com **melhor Dice de todos** (0.9139 no teste, contra

0.8881 do original) e ao mesmo tempo com um dos piores mAP. Contexto global melhora classificacao e nao melhora separacao, medido duas vezes em regimes diferentes.

O modelo final entregue continua sendo o `dsb2018_boundary`, com a decodificacao calibrada. As mudancas testadas ficaram no repo com o numero delas, que e o que da lastro pra afirmar que foram testadas e nao apenas cogitadas.

Parte 4, inferencia em mosaico

O slide 83 descreve a pratica padrao pra imagem grande: processa em tiles com patches sobrepostos, considera a parte interna e faz a media dos resultados. Isso resolve segmentacao semantica.

Por que quebra pra instancia. O id de uma instancia e arbitrario. O nucleo 7 do tile da esquerda e o nucleo 3 do tile da direita podem ser o mesmo nucleo, e nao existe media entre 7 e 3. Media de rotulo nao e uma operacao definida. Pior, um nucleo cortado pela emenda vira dois objetos, cada um com aproximadamente metade da area, e nenhum dos dois casa com o ground truth nem no limiar mais frouxo de IoU 0.50.

Implementamos quatro estrategias (`src/pa1/tiling.py`, rodadas por `scripts/part4_mosaic.py`):

- **full**: sem tiling, imagem inteira de uma vez. E o teto de referencia.
- **per_tile**: decodifica dentro de cada tile e cola so a parte interna, que e o slide 83 aplicado ao pe da letra. E o que a gente mostra quebrando.
- **blend**: faz a media dos logits na sobreposicao e decodifica **uma vez so** no fim, na imagem inteira.
- **fuse**: decodifica por tile e depois costura, unindo instancias de tiles vizinhos que se sobrepoem na faixa comum.

A correcao que o item 4 pede e a **fuse**. O criterio e IoU na faixa de sobreposicao, com limiar 0.25, e a uniao e transitiva via union-find. As duas decisoes tem motivo. IoU e nao "encostou" porque dois nucleos vizinhos de verdade tambem encostam e nao podem ser fundidos, e o que separa os casos e que instancias correspondentes tem IoU alto na faixa comum enquanto vizinhos legitimos tem IoU perto de zero. Union-find porque um nucleo que aparece em tres tiles precisa virar um objeto e nao dois, e uniao par a par sem transitividade deixaria isso passar. Tem teste pros tres casos em `tests/test_tiling.py`: objeto na emenda vira dois pedacos no `per_tile` e um so no `fuse`, dois objetos genuinamente distintos continuam dois, e objeto atravessando tres tiles sai como um.

A **blend** merece um comentario conceitual que vale na apresentacao: ela e a que mais se aproxima do espirito do slide 83, e o unico motivo de ela ser possivel e que a nossa representacao e densa. Da pra fazer media de logit de fronteira porque logit e um numero comparavel entre rodadas. Um detector com proposta de regioao, que e o que o PA proibiu, nao teria esse caminho: nao existe media entre duas caixas propostas em tiles diferentes, so NMS, que e justamente uma forma de fusao como a nossa. Ou seja, a proibicao do enunciado nos empurrou pra representacao que torna o problema do tiling mais facil, nao mais dificil.

O resultado

Mosaico de 3x3 imagens do teste (as 9 mais densas), 768x768, com 419 instancias no ground truth. Rodamos com tres tamanhos de tile de proposito, porque tile menor cria mais emenda: com tile 256 sao 9 tiles, com tile 96 sao 81. Em `results/parte4/`.

tile / sobreposicao	full	per_tile	blend	fuse
256 / 64	0.3512	0.2741	0.3527	0.3374
128 / 32	0.3512	0.2061	0.3476	0.3148
96 / 16	0.3512	0.1751	0.3318	0.2856

E a contagem de objetos, que e onde a falha fica gritante (o ground truth tem 419):

tile / sobreposicao	full	per_tile	blend	fuse
256 / 64	360	461	361	356
128 / 32	360	536	360	348
96 / 16	360	621	356	337

Essa segunda tabela e o slide. Sem tiling o modelo preve 360 objetos, ja subestimando. Com o metodo do slide 83 aplicado ao pe da letra e tile 96, ele preve **621**, um excesso de 48% sobre o ground truth. O modelo nao mudou, a rede e a mesma, os pesos sao os mesmos. Os 261 objetos a mais foram **criados pelo pos-processamento**, sao nucleos cortados pelas emendas e contados duas vezes. E a degradacao e monotonica: quanto menor o tile, mais emenda, mais objeto fantasma e menos mAP (0.274, 0.206, 0.175).

O **blend** e praticamente imune (0.353, 0.348, 0.332, contagem sempre perto de 360) e o **fuse** recupera boa parte (0.337, 0.315, 0.286).

Olhando so nas instancias que efetivamente cruzam uma emenda:

tile	instancias na emenda	IoU medio, per_tile	IoU medio, fuse	recuperadas em 0.50
256	7	0.516	0.606	5 e 5
128	11	0.543	0.521	5 e 6
96	14	0.597	0.657	10 e 12

A costura melhora o IoU medio dessas instancias em dois dos tres casos. No tile 128 ela piora um pouco, e a explicacao e que a fusao por IoU as vezes une dois nucleos vizinhos de verdade que aparecem juntos em dois tiles, o que troca dois objetos certos por um errado. E o preco de decidir primeiro e consertar depois.

Por que blend ganha de fuse. Blend faz a media antes de decidir, entao o watershed roda uma vez so sobre um mapa continuo consistente na imagem inteira, e o problema de identidade de instancia nem chega a existir. Fuse decide primeiro e conserta depois, e conserto depois de uma decisao errada nunca recupera tudo: se o watershed ja cortou um nucleo ao meio dentro de um tile, unir os dois pedacos devolve a area mas nao devolve o contorno que teria saído de uma decisao unica.

A conclusao que vale pra apresentacao e que o slide 83 esta certo, mas o "faca a media dos resultados" precisa ser lido como **media da saida densa da rede, antes de decodificar**, e nao media do resultado final. Pra segmentacao semantica os dois sao a mesma coisa, porque nao existe passo de decodificacao. Pra instancia sao coisas completamente diferentes.

E vale notar a ironia: o unico motivo de o **blend** existir e que a nossa representacao e densa. Um detector com proposta de regioao, que e o que o PA proibiu, nao teria esse caminho, porque nao existe media entre duas caixas propostas em tiles diferentes, so NMS, que e justamente uma forma de fusao como a nossa e sofre do mesmo problema. Ou seja, a proibicao do enunciado nos empurrou pra representacao que torna o problema do tiling **mais facil**, nao mais dificil.

Parte 5, campo receptivo teorico e galeria de falhas

O campo receptivo (item obrigatorio)

A conta e a recorrência padrao dos slides 35 a 38, implementada em `src/pa1/receptive_field.py`. Percorrendo as camadas em ordem, com j o espacamento acumulado e r o campo receptivo:

```
j_out = j_in * s
r_out = r_in + (k - 1) * d * j_in
```

com k o tamanho do kernel, s o stride e d a dilatacao. Comeca em $j = 1$ e $r = 1$. O slide 38 e exatamente isso: pooling aumenta r sem custar parametro, mas cobra em resolucao. O slide 39 mostra a alternativa atrous, que aumenta r sem mexer em j nem no numero de pesos.

Para o encoder do modelo final, ResNet34, por estagio:

estagio	campo receptivo	stride acumulado
stem (conv 7x7 + maxpool)	11 px	4
fim do layer1	59 px	4
fim do layer2	179 px	8
fim do layer3	547 px	16
fim do layer4	899 px	32

E comparando encoders e a variante atrous, todos na mesma conta:

encoder	campo receptivo	output stride	parametros
ResNet34 (o nosso)	899 px	32	24.44M
ResNet34, atrous no layer4	931 px	16	24.44M
ResNet34, atrous no layer3 e layer4	947 px	8	24.44M
ResNet18	435 px	32	
ResNet18, atrous no layer4	467 px	16	

A coluna de parametros e o argumento do slide 39 verificado na pratica: dilatar nao adiciona um peso sequer, o filtro e o mesmo com buraco no meio. O que muda e so a resolucao do mapa de saida. Uma pegadinha de implementacao: o **BasicBlock** do torchvision (que e o bloco da ResNet18 e da ResNet34) recusa

`replace_stride_with_dilation`, so o **Bottleneck** aceita, entao a gente aplicou a dilatacao nas convs na mao em `src/pa1/models/encoders.py`.

A comparacao com o tamanho dos objetos, e a surpresa

Os nucleos do DSB2018 tem diametro equivalente mediano de **19.4 px** no split de teste (4371 instancias), p95 de 47.8 px e maximo de 93.4 px. No treino (9367 instancias) a mediana e 20.7 px, a media 21.9 px e o maximo 87.7 px. Ou seja, os dois splits contam a mesma historia, e o histograma esta em `results/parte5/receptive_field.png`.

Confrontando com a tabela acima, o diagnostico que o proprio enunciado da como exemplo ("o objeto tem 180 px de diametro e o campo receptivo teorico do meu encoder e 140 px") **nao se aplica ao nosso caso, e por uma margem enorme**. O campo receptivo teorico do ResNet34 e 899 px e o maior nucleo do dataset tem 93 px. Nem o maior objeto chega a um decimo do campo receptivo. Se a nossa unica ferramenta de diagnostico fosse campo receptivo, a conclusao seria que nao ha nada errado, e claramente ha.

A coluna que conta a historia certa e a outra: **output stride 32**. O feature map mais profundo tem uma celula a cada 32 px da imagem, e o nucleo mediano tem 19.4 px de diametro. Ou seja, **um nucleo inteiro e menor do que uma unica celula do topo do encoder**, e dois nucleos encostados cabem folgados dentro da mesma celula. No fim do layer4 nao existe representacao nenhuma capaz de distinguir "um nucleo" de "dois nucleos encostados", porque os dois casos produzem exatamente a mesma ativacao naquela resolucao.

Isso reposiciona todo o resto do trabalho. A informacao que separa as instancias so pode vir dos estagios rasos, onde o stride ainda e 4 ou 8, e chega ao decoder pelas skip connections, nao pelo caminho profundo. E e por isso que a nossa previsao pro eixo 3 e de ganho pequeno em mAP: contexto global se pluga no topo do encoder, exatamente na resolucao onde a informacao de separacao ja foi destruida. Contexto global pode ajudar a decidir se aquilo e nucleo ou nao (classificar), mas nao tem como ajudar a decidir onde cortar entre dois (separar).

Isso tambem responde a pergunta do enunciado sobre atrous. Como o campo receptivo ja e grande demais, o valor de atrous aqui **nao e o campo receptivo, e o output stride**: dilatar o layer4 leva o stride de 32 pra 16, e dilatar layer3 e layer4 leva pra 8, que ja e menor que o diametro de um nucleo. O ganho de campo receptivo que vem junto (899 para 931 para 947 no ResNet34) e irrelevante nesse dataset, porque 899 ja era grande demais. Essa e a leitura que a gente quer defender na apresentacao: o mesmo mecanismo do slide 40, mas o beneficio dele aqui e o efeito colateral, nao o efeito anunciado.

A galeria de falhas

As cinco piores imagens do teste estao em `results/parte5/failure_1.png` a `failure_5.png`, cada uma com imagem, ground truth, predicao e os quatro mapas intermediarios (foreground, interior, fronteira e distancia), que e o que o enunciado pede. Os numeros que sustentam o diagnostico de cada uma:

#	imagem	AP	gt	previstas	fundidas	fragmentadas	diametro mediano
1	942d56861f	0.037	51	54	0	0	16 px
2	3a3fee427e	0.091	56	56	2	2	18 px
3	13c8ff1f49	0.112	17	14	2	0	13 px
4	ad473063da	0.113	84	60	18	0	12 px

#	imagem	AP	gt	previstas	fundidas	fragmentadas	diametro mediano
5	358e47eaa1	0.134	52	54	2	2	19 px

O padrao salta aos olhos: **as cinco tem nucleo pequeno**, de 12 a 19 px de diametro mediano, contra 19.4 px da mediana do dataset, e quatro das cinco sao imagens densas, com 51 a 84 nucleos. O caso 1 e o mais instrutivo, porque ele nao tem nenhuma fusao nem fragmentacao (a contagem quase bate, 54 contra 51) e mesmo assim tira AP 0.037. Ou seja, ele achou quase o numero certo de objetos e errou o **contorno** de praticamente todos. Isso e falha de delineamento, nao de separacao, e e um modo de erro diferente do que a Parte 1 tinha.

O caso 4 e o oposto e e o modo de falha classico: 84 nucleos verdadeiros, 60 previstos, 18 instancias previstas cobrindo dois ou mais nucleos de verdade. E o aglomerado denso onde a casca de 2 px entre nucleos simplesmente nao foi prevista.

O diagnostico, com o numero do erro no split inteiro

Somando os quatro modos de erro nas 101 imagens de teste, nos dois modelos

([results/parte5/results.json](#) pra trilha A e [results/parte5/baseline_results.json](#) pro baseline):

modo de erro	Parte 1 (limiar + CC)	Parte 2 (fronteira + watershed)
fusoes (uma previsao cobre 2+ nucleos)	582	260
fragmentacoes (2+ previsoes num nucleo)	4	67
nucleos nao achados	1505	886
previsoes espurias	510	550

Essa tabela e a versao quantitativa do que a Parte 2 prometeu. A trilha A **cortou as fusoes em 55%**, de 582 pra 260, que era exatamente o objetivo. E o preco esta na linha de baixo: a fragmentacao, que praticamente nao existia na Parte 1 (4 casos), subiu pra 67. E o modo de falha novo que a representacao introduz, quando o interior previsto racha em dois marcadores e o watershed corta um nucleo saudavel ao meio. Trocamos 322 fusoes por 63 fragmentacoes, o que e um bom negocio, mas nao e de graca.

A correcao, e o que ela revelou

O diagnostico da secao anterior diz que o problema nao e campo receptivo, e resolucao: com output stride 32 um nucleo de 19 px cabe dentro de uma celula do mapa mais profundo. A mudanca que isso sugere e direta e e o mecanismo do slide 40: dilatar o layer4 pra levar o output stride de 32 pra 16, sem adicionar parametro nenhum. Esta em [configs/dsb2018_boundary_os16.yaml](#), e o antes/depois no mesmo split de teste:

metrica	output stride 32	output stride 16	mudou
mAP @[.50:.95]	0.4972	0.4830	pior
AP @.50	0.7592	0.7742	melhor
AP @.90	0.157	0.121	pior
Dice	0.8881	0.8857	igual

metrica	output stride 32	output stride 16	mudou
erro de contagem	4.10	3.77	melhor
fusoes	260	251	melhor
nucleos nao achados	886	798	melhor
fragmentacoes	67	71	pior

A correcao funcionou no que o diagnostico previa e falhou no total. Todas as medidas de separacao melhoraram: AP no limiar frouxo subiu 1.5 ponto, o erro de contagem caiu 8%, as fusoes cairam e os nucleos nao achados cairam 10%. Mas o mAP agregado caiu 0.014, porque os limiares severos (0.90 e 0.95) pioraram.

O que isso revela, e e a parte interessante: o diagnostico estava **certo sobre o mecanismo e incompleto sobre a consequencia**. Ele previa que mais resolucao no topo do encoder ajudaria a separar, e ajudou. O que ele nao considerou e que dilatar o layer4 introduz o efeito de gridding do atrous (o filtro passa a amostrar pixels alternados, e pixels vizinhos passam a ser processados por conjuntos disjuntos de pesos), o que degrada o contorno fino. Separacao melhora, delineamento piora, e como o mAP media dez limiares de IoU, sendo metade deles severos, o delineamento domina o agregado.

Vale notar que esse e o **terceiro** lugar do trabalho onde aparece o mesmo trade-off entre separar e delinear: a Parte 2 contra a Parte 1, o alpha do eixo 2, e agora o output stride. Nos tres casos a mesma tensao, e nos tres a metrica agregada esconde o que esta acontecendo. Se o objetivo fosse contar celulas, as tres decisoes seriam tomadas na direcao oposta.

Parte 6, teste de estresse por corrupcao

Escolhemos a opcao das corrupcoes, entre as tres oferecidas, porque e a unica que produz uma curva de degradacao de verdade, com eixo x ordenado, em vez de dois pontos soltos. Sao tres familias em tres intensidades cada ([src/pa1/corruptions.py](#)):

corrupcao	intensidade 1	2	3
blur gaussiano	sigma 1.0	2.0	4.0
ruido gaussiano	desvio 8	20	40
brilho e contraste	ganho 0.75, vies -10	0.55, -25	0.35, -40

As corrupcoes entram so na avaliacao, nunca no treino, tirando o brilho e contraste leve que ja esta no augment. Essa assimetria e o ponto do teste.

A tabela reporta Dice do lado do mAP de proposito, pela mesma logica do eixo 3: se o Dice cai junto, o modelo esta perdendo a capacidade de achar o objeto, e se o Dice segura e so o mAP cai, ele ainda ve os nucleos mas perdeu a capacidade de separa-los, o que localizaria o dano na cabeca de fronteira e nao no reconhecimento.

A previsao a confrontar: blur deveria ser o pior dos tres, porque a classe fronteira e uma casca de 2 px e um blur de sigma 4 destroi literalmente a estrutura que a rede precisa prever. Ruido e brilho nao atacam a geometria, so o contraste.

O resultado

Split de teste inteiro, modelo da trilha A. Em [results/parte6/results.json](#) e a curva em [results/parte6/degradation.png](#):

corrupcao	intensidade	mAP	Dice	erro de contagem
nenhuma	0	0.4972	0.8881	4.10
ruido	1	0.5145	0.8990	4.37
ruido	2	0.4328	0.8725	4.93
ruido	3	0.3286	0.8312	5.75
blur	1	0.4265	0.8582	4.16
blur	2	0.2594	0.7536	5.13
blur	3	0.1731	0.6346	8.35
brilho e contraste	1	0.4452	0.8701	4.39
brilho e contraste	2	0.2412	0.6692	10.15
brilho e contraste	3	0.0931	0.3499	24.06

Tres coisas, e as duas primeiras contrariam o que a gente tinha previsto.

Ruido leve melhora o modelo. Com intensidade 1 o mAP sobe de 0.4972 pra 0.5145 e o Dice de 0.8881 pra 0.8990. Nao e ruido de medicaao, sao 101 imagens e o efeito aparece nas duas metricas. A explicacao e que [A.GaussNoise](#) esta no augment de treino, entao ruido leve e dentro da distribuicao que a rede viu, e adicionar um pouco funciona como leve regularizacao na inferencia. E um lembrete util: "corrupcao" nao e sinonimo de "pior", o que importa e a distancia pra distribuicao de treino, nao a degradacao perceptual.

Blur nao e o pior, brilho e contraste e. A gente tinha previsto blur como o pior, porque a classe fronteira e uma casca de 2 px e um blur de sigma 4 destroi a estrutura que a rede precisa prever. Blur 3 de fato derruba pra 0.1731, mas brilho e contraste 3 derruba pra **0.0931**, quase o dobro de dano. O motivo aparece na coluna do Dice: com brilho 3 o Dice desaba pra 0.3499, ou seja, o modelo perde o objeto inteiro, nao so a separacao. Ganho 0.35 com vies -40 achata a imagem quase toda numa faixa estreita de cinza escuro, e o encoder pre-treinado na ImageNet nunca viu nada assim. O augment de treino tem brilho e contraste, mas com limite 0.25, muito mais suave que os 0.65 da intensidade 3.

A terceira leitura e a que responde a pergunta de projeto, e sai de olhar mAP e Dice juntos, que e o motivo de a tabela ter as duas colunas:

corrupcao intensidade 3	queda do Dice	queda do mAP	razao
ruido	-6%	-34%	5.6x
blur	-29%	-65%	2.3x
brilho e contraste	-61%	-81%	1.3x

Com ruído o Dice quase não se mexe (cai 6%) e o mAP cai 34%. Ou seja, sob ruído o modelo **continua enxergando os núcleos e perde a capacidade de separá-los**. Isso localiza a fragilidade exatamente onde a gente esperava: na cabeça de fronteira, que precisa acertar uma casca de 2 px e é por construção a parte mais fina e mais sensível da representação. Com brilho e contraste a razão vai pra 1.3, ou seja, ali o dano é generalizado, o modelo perde tudo junto.

O custo prático dessa fragilidade está na última coluna da tabela grande: com brilho e contraste 3 o erro de contagem vai de 4 pra **24 núcleos por imagem**, que é mais do que a Parte 1 errava na imagem limpa.

O que ficou de fora, e o que a gente faria com mais tempo

Vale ser explícito sobre os limites do que está aqui, porque na apresentação alguém vai perguntar.

O eixo 1 da Parte 3 não foi rodado. Escolhemos os eixos 2 e 3, que é o que o enunciado pede (dois dos três). A comparação entre pool indices, skip connections e atrous ficou de fora como experimento controlado, embora a análise de campo receptivo da Parte 5 e a correção com output stride 16 toquem no mesmo assunto por outro caminho.

A busca de hiperparametro foi mínima. Learning rate, weight decay, tamanho do crop e número de épocas foram escolhidos de uma vez e não variados. O único ajuste feito com método foi a decodificação do watershed, varrida na validação com `scripts/tune_watershed.py`, e a espessura da fronteira, escolhida pela tabela de marcadores perdidos e rachados. Tudo mais é chute informado.

Uma única seed no modelo final. As 2 seeds exigidas estão nas ablações da Parte 3. O modelo final da Parte 2 rodou com seed 0 só, então a diferença de 0.0437 de mAP entre Parte 1 e Parte 2 não vem com barra de erro. O que dá confiança de que o efeito é real é que ele aparece com o mesmo sinal e muito maior no dataset sintético, e que o erro de contagem cai 60%, que é uma diferença grande demais pra ser ruído de seed.

O teste foi olhado mais de uma vez. A gente selecionou checkpoint pela validação e calibrou o watershed pela validação, o que está certo, mas rodou a avaliação de teste várias vezes ao longo do desenvolvimento. Não houve escolha de modelo feita pelo número de teste, mas registrar isso é mais honesto do que fingir que o teste foi aberto uma vez.

As ablações rodaram num regime que não é o do modelo final, e a gente só descobriu o tamanho disso no fim, quando levou a conclusão do eixo 2 de volta pro orçamento cheio e ela não se sustentou. O certo teria sido rodar as ablações com 40 épocas, o que custaria umas 2 horas de GPU em vez de 53 minutos, ou pelo menos fixar o schedule em vez de deixar o T_max seguir o número de épocas. Ficou como está, documentado, porque o resultado negativo também ensina.

Com mais tempo, na ordem em que a gente atacaria: refazer o eixo 2 com 40 épocas pra ver se o ranking muda mesmo ou se foi só o schedule, rodar o modelo final com 3 seeds pra ter barra de erro na comparação principal, tentar a trilha B pra ver se embedding separa melhor que fronteira nos aglomerados densos do cluster 3, e treinar com output stride 8 pra levar o diagnóstico da Parte 5 até o fim.

Mapa dos entregáveis

item do enunciado	onde está
repositorio com historico	commits ao longo do desenvolvimento, não um só

item do enunciado	onde esta
README com ambiente, dados, um comando que treina, um que avalia	README.md
AI_LOG	AI_LOG.md
notebook de inferencia que roda sem retreinar	inferencia.ipynb , logica em src/pa1/inference.py
checkpoint do modelo final	runs/dsb2018_boundary/best.pt , link no README
Parte 3 aplicada de volta no modelo final	configs/dsb2018_final*.yaml , resultado negativo documentado
Parte 0, gerador sintetico e teste em menos de 5 min	src/pa1/data/synthetic.py , configs/synthetic*.yaml
Parte 1, baseline binario, IoU e Dice	configs/dsb2018_baseline.yaml , results/parte1/
Parte 1, instancias por limiar e componentes conexos	src/pa1/postprocess/naive.py
Parte 1, mAP com matching proprio	src/pa1/metrics/instance.py , tests/test_instance_metrics.py
Parte 1, regra de matching documentada	secao da metrica aqui, e as duas regras medidas
Parte 1, grafico contra densidade	results/parte1/density.png
Parte 2, trilha A	src/pa1/data/targets.py , src/pa1/postprocess/watershed.py
Parte 2, metricas lado a lado	scripts/compare.py , results/parte2/comparacao/
Parte 3, eixos 2 e 3, 2 seeds	scripts/ablations.py , results/parte3/
Parte 4, mosaico, falha e correcao	src/pa1/tiling.py , results/parte4/
Parte 5, campo receptivo e galeria	src/pa1/receptive_field.py , results/parte5/
Parte 5, a correcao	configs/dsb2018_boundary_os16.yaml
Parte 6, corrupcoes	src/pa1/corruptions.py , results/parte6/

Como reproduzir tudo

Os comandos estao no README, na secao "Reproduzir cada parte". Em GPU o pipeline inteiro das Partes 0, 1, 2, 4, 5 e 6 leva em torno de 30 minutos, e a Parte 3 leva algumas horas porque sao 18 treinos. Em CPU tudo roda igual, so muito mais devagar.

Nenhuma das nossas maquinas tem GPU NVIDIA, entao os treinos sairam de kernel do Kaggle com o notebook de [notebooks/colab_setup.ipynb](#) adaptado. Uma pegadinha que custou tempo: o Kaggle as vezes entrega uma P100, que e sm_60, e o PyTorch da imagem deles so cobre sm_70 pra cima, entao [torch.cuda.is_available\(\)](#) da True e nada roda. O notebook checa [torch.cuda.get_device_capability\(\)](#) no inicio e instala um torch compativel se precisar.